CSYE 6300 • SUMMER 2026 • GROUP 3

CloudPulse

Design Document — Milestone 2

Project: Cloud Service Health Monitoring and Alert Dashboard

Team: Katha Patel • Ashwin S. Thankachan • Meet Patel • Pavithra Prasad

Repository: github.com/KathaPatel29/csye6300-summer26-group3-final-project

Document type: Code-free technical blueprint based on the current repository state

1. Purpose and Scope

CloudPulse is a student-scale service-health monitoring system. It registers application services, periodically or manually checks their health endpoints, normalizes different response formats, evaluates availability and latency, tracks lifecycle changes, and creates alerts.

At Milestone 2, the design is centered on a Spring Boot backend and three demonstrable monitored services. The system contains no cache component at this milestone.

2. Design Patterns Used

The pattern list remains exactly the nine patterns selected in Milestone 1. MVC is an architectural pattern rather than a Gang of Four pattern.

Pattern	Problem solved	Real classes involved
Factory Method	Creates HTTP or mock monitors without making callers depend directly on concrete construction.	MonitorCreator; HttpMonitorCreator; MockMonitorCreator; ServiceMonitor; HttpServiceMonitor; MockServiceMonitor
Adapter	Normalizes incompatible standard and legacy health-response shapes into one internal result.	HealthResponseAdapter; StandardHealthAdapter; LegacyHealthAdapter; HealthCheckResult; HttpServiceMonitor
Decorator	Adds retry, logging, or metrics behavior without modifying the underlying monitor.	MonitorDecorator; LoggingMonitorDecorator; RetryMonitorDecorator; MetricsMonitorDecorator; ServiceMonitor
Strategy	Allows services to use different latency thresholds within the same evaluation workflow.	HealthEvaluationStrategy; NormalHealthStrategy; StrictHealthStrategy; HealthEvaluationService; HealthCheckSnapshot

Pattern	Problem solved	Real classes involved
State	Makes the next lifecycle status depend on the current state and observed health, including explicit recovery.	ServiceState; ServiceStateMachine; UnknownState; HealthyState; DegradedState; DownState; RecoveredState
Chain of Responsibility	Separates validation, availability, latency, status-change, and alert processing into ordered steps.	ResultHandler; ResponseValidationHandler; AvailabilityHandler; LatencyHandler; StatusChangeHandler; AlertCreationHandler; HealthCheckProcessor
Command	Represents checks, monitoring toggles, acknowledgement, and resolution as consistently executable actions.	MonitoringCommand; RunHealthCheckCommand; EnableMonitoringCommand; DisableMonitoringCommand; AcknowledgeAlertCommand; ResolveAlertCommand
Observer	Decouples status-event producers from dashboard, alert, and history reactions.	EventSubscriber; StatusEventPublisher; DashboardUpdateObserver; AlertObserver; EventHistoryObserver; StatusEvent
MVC	Separates REST request handling, application data, and the planned user-facing dashboard.	ServiceController; AlertController; RegisterServiceRequest; AlertResponse; domain entities; repositories; planned React View

3. System Architecture

3.1 Major Components

Component	Responsibility	State
React View	Presents service health, registration controls, alerts, and operator actions.	Planned
Spring Boot REST backend	Provides service and alert APIs and coordinates application operations.	Implemented
Monitoring engine	Runs HTTP or mock checks, applies wrappers, and normalizes health responses.	Implemented
Evaluation and lifecycle	Applies the selected policy and determines lifecycle transitions.	Implemented
Processing and alert workflow	Validates results, records changes, and creates or resolves alerts.	Implemented
Scheduler	Loads enabled services and starts periodic check commands.	Implemented
Observer module	Notifies dashboard, alert, and history subscribers.	Partial
Persistence layer	Stores services, results, status events, and alerts through JPA.	Implemented
PostgreSQL	Target persistent database for the composed deployment.	Partial
H2	Current in-memory development and test database.	Implemented
Three demo services	Expose standard, legacy, and flaky health behavior.	Implemented

3.2 Architecture Diagram

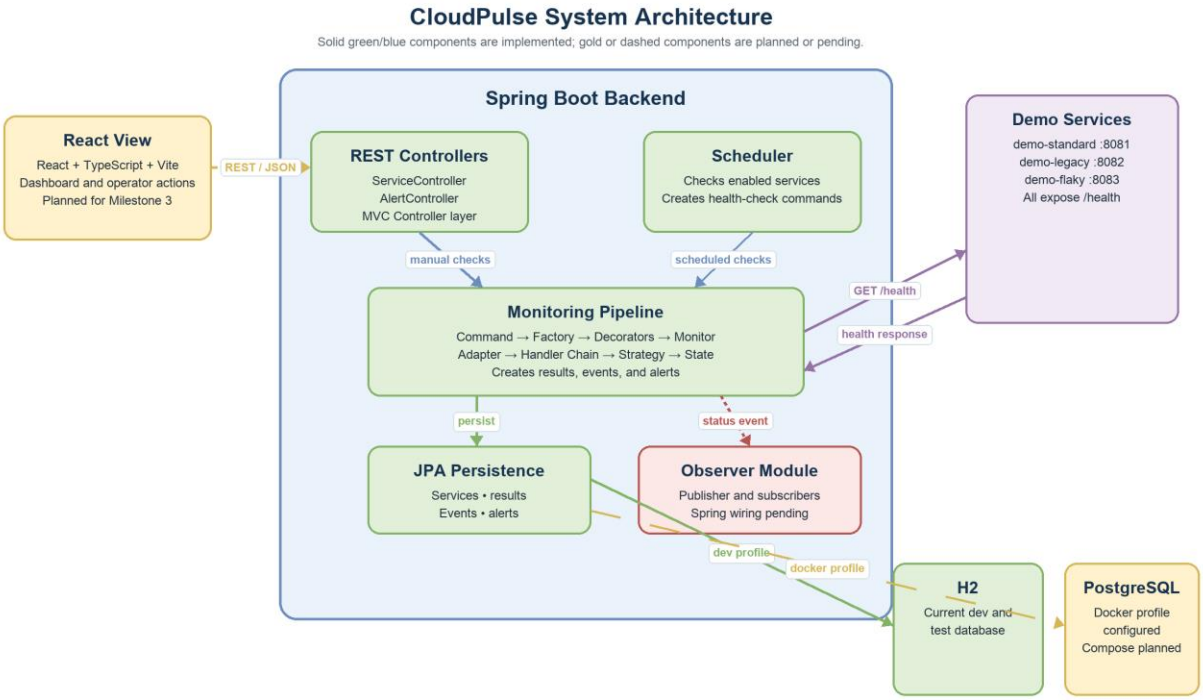

Figure 1. CloudPulse component boundaries and principal data-flow directions.

3.3 Component Interaction and Data Flow

Frontend → Backend via REST. The planned React View sends registration, check, monitoring-toggle, acknowledgement, and resolution requests. The backend returns JSON service and alert data.

Scheduler → Monitoring workflow. HealthCheckScheduler loads enabled services and creates a RunHealthCheckCommand for each service.

REST controller → Monitoring workflow. ServiceController can start the same check operation immediately for a manual request.

Backend → Demo services via HTTP. HttpServiceMonitor sends an HTTP GET request to a registered health URL.

Demo services → Monitoring engine. The service returns an HTTP status and response body. A compatible adapter converts it into a HealthCheckResult.

Monitoring result → Processing chain. RunHealthCheckCommand sends the result through validation, availability, latency, status-change, and alert handlers.

Processing chain → Strategy and State. LatencyHandler selects the evaluation policy; StatusChangeHandler delegates lifecycle behavior to ServiceStateMachine.

Processing workflow → Persistence. Repositories store MonitoredService, HealthCheckResult, StatusEvent, and Alert data in H2 or the configured PostgreSQL database.

Persistence → Backend → Frontend. Controllers read current data and return it to the planned React View through REST.

Status event → Observer subscribers. The intended flow notifies dashboard, alert, and history observers; this runtime wiring remains pending.

4. Component and Service Breakdown

Package or service	Purpose	Patterns used and rationale
controller/	REST operations for services and alerts.	MVC, Command, Factory Method — controllers handle requests, delegate actions, and select the monitor creator.
monitor/	Monitoring interface, HTTP and mock monitors, creators, and wrappers.	Factory Method and Decorator — construction varies by type while extra behavior is layered.
adapter/	Converts standard and legacy response bodies into health results.	Adapter — incompatible external formats become one internal representation.
scheduler/	Periodically starts checks for enabled services.	Command, Factory Method, Decorator — scheduled work creates a command around a selected, wrapped monitor.
evaluation/	Applies NORMAL or STRICT health rules.	Strategy — thresholds vary without changing the caller or processing chain.
lifecycle/	Applies status-transition behavior.	State — transition behavior depends on the service's current lifecycle state.
handler/	Processes results through ordered stages.	Chain of Responsibility, Strategy, State — each stage owns one task and delegates specialized decisions.
command/	Encapsulates checks, monitoring toggles, acknowledgement, and resolution.	Command — controllers and the scheduler invoke consistent action objects.
observer/	Defines publication and dashboard, alert, and history subscribers.	Observer — event producers do not need direct dependencies on every reaction.
model/	Defines persistent domain entities, statuses, policies, types, and severity.	MVC Model role — represents stored data and controller responses.
repository/	Provides persistence operations for the JPA entities.	MVC Model/persistence role — isolates stored-data access.
demo-standard	Predictable standard-format health endpoint.	Adapter integration target — interpreted by StandardHealthAdapter.
demo-legacy	Legacy-format health endpoint.	Adapter integration target — interpreted by LegacyHealthAdapter.
demo-flaky	Healthy, slow, and failed behavior for demonstrations.	Exercises Strategy, State, Chain of Responsibility, and alert handling.
Planned React frontend	Displays health and alerts and sends operator actions.	MVC View role — the user-facing View for the Spring controllers.

5. Technology Stack

Layer	Technology	Use and current state
Language	Java 22	Backend and all three demo services — implemented
Backend framework	Spring Boot 3.3.4	Bootstrap, dependency injection, configuration, and testing — implemented
Web API	Spring Web	REST controllers and demo health endpoints — implemented

Layer	Technology	Use and current state
Scheduling	Spring Scheduler	Periodic checks of enabled services — implemented
Validation	Jakarta Validation	Service-registration request validation — implemented
Persistence	Spring Data JPA and Hibernate	Entity mapping and repository access — implemented
Development database	H2	Default in-memory development and test database — implemented
Target database	PostgreSQL	Persistent Docker deployment database — profile and driver configured
Response parsing	Jackson	Parses standard and legacy JSON responses — implemented
API documentation	Springdoc OpenAPI 2.6.0	Swagger UI and API documentation — dependency included
Build	Maven and Maven Wrapper	Builds and tests backend and demo services — implemented
Testing	JUnit 5, Mockito, Spring Boot Test, MockMvc	Unit, controller, and workflow tests — implemented
Continuous integration	GitHub Actions	Backend verification and conditional frontend build — implemented
Frontend	React, TypeScript, Vite	Dashboard and MVC View — planned
Infrastructure	Docker Compose	Backend, PostgreSQL, and demo-service orchestration — planned

6. Milestone 3 Architecture Completion

Milestone 3 will complete the planned system boundary by adding the React View, Docker Compose deployment, expanded integration coverage, Observer runtime wiring, and scheduling refinement.

- Create the React, TypeScript, and Vite dashboard and connect it to ServiceController and AlertController through REST.
- Add docker-compose.yml for the backend, PostgreSQL, and the three demo services.
- Extend integration coverage to real HTTP calls against the standard, legacy, and flaky services and to PostgreSQL-backed execution.
- Register the Observer publisher and subscribers with Spring and publish saved lifecycle events from the processing workflow.
- Apply each service’s checkIntervalSeconds value instead of relying only on one global delay.
- Expose any additional read operations required for persisted health-check and status-event history in the dashboard.